INDIAN INSTITUTE OF TECHNOLOGY
PALAKKAD

MADHAVA USER MANUAL



IIT PALAKKAD

BASIC LINUX COMMANDS

1. NAVIGATING THE FILE SYSTEM

- **pwd** - Print the current working directory.
- **ls** - List the contents of a directory.
  - `ls -l # Detailed listing such as permissions, owner, size, etc.,`
- **cd** - Change the directory.
  - `cd /path/to/directory # Navigate to a specific directory.`
  - `cd .. # Go up one directory level.`
  - `cd ~ # Go to the home directory.`

2. FILE OPERATIONS

- **cp** - Copy files or directories.
  - `cp myfile.dat ~/ # Copy a myfile.dat to home directory.`
- **mv** - Move or rename files or directories.
  - `mv myfile.dat yourfile.dat # Rename a file or directory.`
- **touch** - Create an empty file or update the timestamp of a file.
- **cat** - Display the contents of a file.
- **less** - View the contents of a file page by page (good for large files).

3. SEARCHING FOR FILES

- **find** - Search for files and directories.
  - `find /path/to/search -name "filename" # Find a file by name.`
  - `find /path/to/search -type f -name "*.txt" # Find .txt files.`
- **locate** - Find files by name (faster than find as it uses a database).
  - `locate file1.so #Locates the file named file1.so`
- **grep** - Search for a pattern in a file.
  - `grep "pattern" filename # Find occurrences of a pattern in a file.`
  - `grep -r "pattern" directory # Search recursively in a directory.`

4. MANAGING PROCESSES

- **top** - Provides a dynamic, real-time view of system processes.
- **ps** - View running processes.
  - `ps aux # Show all processes running on the system.`
- **kill** - Terminate a process by its PID.
  - `kill 414243 #Kill the process with PID 414243`
- **killall** - Terminate processes by name.
  - `killall proc1 #Kill the process named proc1.`

For further details on any command, you can look up the manual by using the man command (Eg., man grep)

IIT PALAKKAD

BASIC LINUX COMMANDS

5. FILE PERMISSIONS

- **chmod** - Change file permissions.
  - **chmod 755 file.dat** # Set read, write, and execute permissions for the owner and read and execute for others for file.dat
- **chown** - Change file owner.
  - **chown user:group file.dat** # Change the owner of a file.

6. NETWORKING

- **ping** - Test network connectivity.
  - **ping madhava.iitpkd.ac.in** # Ping the master node of Madhava
- **ssh** - Securely connect to a remote system.
  - **ssh username@mach1** # Connect to a remote machine mach1.
  - **ssh -p 49122 username@mach1** #Connect using a port number.
- **scp** - Securely copy files between hosts.
  - **scp file.dat username@mach1:/path/to/destination** #Copies file.dat from current machine to mach1.
  - **scp -P 49122 file.dat username@mach1:~/** # Securely copy files using port number.
- **rsync** - Efficiently sync files between systems.
  - **rsync -av dataset1 dataset2** # Synchronize files.
  - **rsync -av -e "ssh -p 49122" folder username@mach1:~/** # Copies the directory using port number to home of mach1.

7. ENVIRONMENT MANAGEMENT

- **module** - Manage software environments
  - **module avail** # List available modules.
  - **module load gnu8** # Load a module related to gnu compiler.
  - **module unload gnu8** # Unload a module gnu8.
  - **module purge** # Remove all loaded module from user environment.
- **export** - Set environment variables.
  - **export VAR=10** # For current session variable VAR is set to 10.

8. COMPRESSING AND ARCHIVING FILES

- **tar** - Archive files or directories.
  - **tar -czvf archive.tar.gz directory** # Compress a directory.
  - **tar -xzvf archive.tar.gz** # Extract a tarball.
- **gzip** - Compress files.
  - **gzip file.dat** # Compress the file.dat with gzip.
  - **gunzip file.dat.gz** – Decompress a file.

For further details on any command, you can look up the manual by using the man command (Eg., man grep)

IIT PALAKKAD

STORAGE STRUCTURE AND QUOTAS

- **/home (Total : 175 TB)** - Quota of 50 GB per user, Weekly automatic Backup, Backup retention for 30 days
- **/scratch (Total : 350 TB)** - No disk quota, non backed-up storage, Contains special directories for optimal utilisation of disk space.
  - **/scratch/<PI username>/common** # PI can share common files with group members. Group member also has **/scratch/<username>/common**
  - **/scratch/common** # Users can put files to share with all other users. Contains directories "datasets", "packages", "models".

BASIC SLURM COMMANDS

SLURM: Open-source, fault-tolerant, highly scalable cluster management and job scheduling system. It manages job allocation, execution and resource arbitration.

- **sinfo** : display characteristics of partitions

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
standard  up     3-00:00:00     36   idle cn[01-36]
nht       up     3-00:00:00     16   idle cn[37-52]
gpu       up     12:00:00        4   idle gpu[01-04]
hm        up     3-00:00:00      4   idle hm[01-04]
```

  standard **-** Job will be submitted to HyperThreading (HT) enabled nodes
  nht - Job will be submitted to HyperThreading (HT) disabled nodes
  gpu - Job will be submitted to GPU nodes
  hm - Job will be submitted to High Memory (1TB RAM) nodes
- **sbatch** : submit job scripts
  - **sbatch submit.sh** # Submits the job given within submit.sh
- **squeue** : display job information (jobid, partition, name, username, status, time and node)
  - **squeue --me** # Display only the current user's job information
- **scancel** : cancel a job or set of jobs.
  - **scancel 42** # Cancel the job with jobid 42
- **scontrol** : display and changes characteristics of jobs, nodes, partitions.
  - **scontrol show job 42** # Shows a wide range of information including submission time, start time, end time etc.,
- **sacct** : display job accounting data
  - **sacct -j 42 --format=JobID,Elapsed** # Show the time taken by the completed job with id 42

For further details on SLURM commands, you can look up https://slurm.schedmd.com/documentation.html or man pages

SLURM SCRIPTS

A SLURM script is used to submit and manage jobs on a cluster, specifying resources like CPU, memory, and time limits. Loading the necessary packages is essential for successful job execution.

LOADING PACKAGES

To load package one can use one of the following methods depending on the user's requirements.

1. Using module command - Details of which is provided in Basic Linux Commands section 7. The software modules necessary for your SLURM job should be mentioned in the SLURM script.

- **module avail** #Lists around 40 software modules. A few example of loading the module is given below.
- **module load g16 gaussianLinda-16.0** # Load GAUSSIAN and LINDA
- **module load intel-2023 intel-2024 fftw-2.1.5-intel** # Load Intel compilers and FFTW compiled with Intel

Loading the module, makes the packages available to execute (without specifying the full path) on nodes and hence included in the SLURM script.

2. Using SPACK - Spack package manager simplifies the process of installing and managing scientific software, allowing users to install different versions and configurations of the same software, making it easier to experiment with various settings and dependencies. Using Spack is as follows

- **module load spack/1.0.0.dev0** #Load the Spack module
- **source /home/apps/spack/share/spack/setup-env.sh** #Setup the Spack module
- **spack find -lvd lammps** #Query installed package named lammps, with output including a unique hashkey (Eg., txjgnnq lammps@20240829.1)
- **spack load txjgnnq** #Loads the package lammps
- **spack find --loaded** #Provides information on loaded applications

Package loading commands has to be included in the SLURM script.

3. Using Conda - For data science, machine learning, and scientific computing which uses Python, to avoid conflicts between different libraries or versions,

- module load anaconda3-2024 #Loads the Conda package
- source /home/apps/compilers/anaconda3/2024/bin/activate #Setup Conda module
- conda create --name myenv #Create a Conda environment
- conda activate myenv #Activate the Conda environment
- conda install <package-name> #Install Python package

Package loading commands has to be included in the SLURM script.

IIT PALAKKAD

SLURM SCRIPTS

SLURM scripting allows users to specify resources, runtime, and job parameters in a script format.

SAMPLE SLURM SCRIPT

The below example script (*submit.sh*) has the essential features and is self explanatory. Modifications can be made according to user's requirement.

```
#!/bin/bash
#SBATCH --job-name=MPIJob          # Job name
#SBATCH --output=%j.out            # Standard output file (JOBID.out)
#SBATCH --error=%j.err             # Standard error file (JOBID.err)
#SBATCH --time=48:00:00            # Maximum runtime (48 hours)
#SBATCH --mem=256G                 # Total memory (256 GB)
#SBATCH --ntasks=96                # Total number of tasks (96 tasks)
#SBATCH --nodes=1                  # Number of nodes (1 node)
#SBATCH --partition=standard       # HT enabled partition
#SBATCH --begin=now+01:00          # Delayed start of 1 hour

cd $SLURM_SUBMIT_DIR               #ensure that the job runs in the
same directory where it was submitted
module load openmpi/4.1.6          # Load the OpenMPI module
mpirun -np $SLURM_NTASKS ./a.out # Run the parallel job using mpirun
```

CHOICE OF PARTITION

- **standard** - Jobs which run on CPUs, which are not adversely affected by Hyper-Threading(HT) and which require less than 256 GB RAM.
- **nht** - Jobs which run on CPUs, which are require HT disabled and which require less than 256 GB RAM. Note in this partition --ntasks should be less than or equal to 48.
- **hm** - Jobs which run on CPUs, which require more than 256 GB RAM.
- **gpu** - Jobs which run on GPU. There are 2 GPUs per node. Additional tag has to be provided, which is as follows
  - `#SBATCH --gres=gpu:1` # Request 1 GPU

JOB SUBMISSION AND MONITORING

- **Submitting Job**
  - `sbatch submit.sh`
- **Monitoring the status of job (Outputs the jobid, partition, nodes etc.,)**
  - `squeue --me`
- **Killing a job**
  - `scancel jobid`

Sample SLURM scripts are provided in /scratch/common

IIT PALAKKAD

BEST PRACTICES

- **Never submit jobs directly on Master node or Compute nodes.**
- **Compile on Master node.**
- **Use SLURM scheduler for all job submission.**
- **Be Resource-Conscious :** Use appropriate partitions on SLURM and respect shared resources.
- **Use /scratch for all jobs :** Large, non-backed-up storage. If the user deems it necessary, they should back up the data.
- **/home is backed-up regularly**

Misuse of the facility will result in access being revoked



